Project for Fourth Semester of Bachelor of Information Technology

Inventory Management System



Submitted By

Khum Bahadur Bista (341695)

Prajum Bhaila (341702)

Shilbiya Rai (341715)

KIST College of Information Technology

Submitted To

Faculty of Science and Technology

Purbanchal University

October 2026

DECLARATION

We, the undersigned, hereby declare that the project entitled **"Inventory Management System"**, submitted in partial fulfillment of the academic requirements for the Fourth Semester of the Bachelor of Information Technology (BIT) program under Purbanchal University, is the result of our own original work.

We certify that this project has been developed independently and has not been copied, plagiarized, or reproduced from any previously published, submitted, or copyrighted work. Furthermore, this project has not been submitted to any other institution or organization for academic or professional evaluation. All references, sources of information, and contributions from others have been appropriately acknowledged.

We fully understand the importance of academic integrity and accept that any breach of these principles may result in disciplinary action, including the rejection or disqualification of this project.

Name of Student	Registration Number	Signature
Khum Bahadur Bista	058-3-2-02012-2024	
Prajum Bhaila	058-3-2-02019-2024	
Shilbiya Rai	058-3-2-02032-2024	

Date:13/10/2026

RECOMMENDATIONS

This is to certify that the project work titled "**Inventory Management System** ", submitted by **Khum Bahadur Bista, Prajum Bhaila**, and **Shilbiya Rai**, students of the **Bachelor of Information Technology (BIT)** program at KIST College of Information Technology, has been completed as per the requirement of the Fourth Semester of the BIT curriculum under Purbanchal University.

To the best of my knowledge, the submitted project is a genuine and original academic work carried out by the students with honesty and academic integrity. I, therefore, recommend this project for acceptance and official recognition by the University.

Deepak Khadka
Coordinator, BIT
KIST College of Information Technology
Date : / / 2026

CERTIFICATE

This is to certify that **Khum Bahadur Bista, Prajum Bhaila,** and **Shilbiya Rai,** students of Bachelor of Information Technology (BIT) at KIST College of Information Technology, have successfully completed their academic project titled "**Inventory Management System**" as part of the course requirements for the Fourth Semester under the curriculum prescribed by Purbanchal University.

The project work was carried out during the academic session September 2026 - October 2026 and is a part of the continuous practical learning approach promoted by the BIT program. The students demonstrated efforts in the planning, development, and execution of the project, adhering to academic and technical standards.

Dhiraj Kumar Jha	**Signature**	**Date**
External		

Bhola Kafle	**Signature**	**Date**
Project Supervisor		

Deepak Khadka	**Signature**	**Date**
Coordinator, BIT		

ACKNOWLEDGEMENTS

We would like to express my sincere gratitude to Mr**.** Deepak Khadka, Coordinator of Bachelor of Information Technology at KIST College of Science and Technology and our project supervisor for the continuous guidance, technical support, and encouragement provided throughout the development of this project.

We are deeply grateful to our faculty members and lab instructors for sharing their knowledge and guidance on Java programming, database management, Java Swing, and JDBC connectivity, which provided the technical foundation for the development of this project.

We would also like to thank our classmates and friends who tested the **"Inventory Management System"** They helped us identify and resolve software and database issues and provided constructive feedback that significantly improved the final system.

Finally, we extend our heartfelt appreciation to our family for their unwavering support and patience during the long hours spent designing, coding, and refining this project.

Thank You

ABSTRACT

The Inventory Management System is a Java desktop application developed to help small shops record and control their stock and sales without relying on paper registers or spreadsheets. The system stores users, categories, products, customers and orders in a MySQL database and provides a simple Swing graphical interface for working with them.

A user logs in with an e-mail and password and sees the menu according to the role (Super Admin or Admin). The system provides modules for managing users, categories, products and customers with input validation using regular expressions. In the order module the user selects a customer and products, adds them to a cart, and the system checks the stock, calculates the total, reduces the stock, saves the order and generates a PDF bill using the iText library. Past orders can be viewed customer-wise and their bills re-opened.

The project was developed using the Waterfall model and is documented with use case, class, sequence, activity, ER and architecture diagrams. It demonstrates the use of classes and inheritance, packages, Swing event handling, exception handling, file output and JDBC with PreparedStatement. The system can be extended with password hashing, search and reports, and a web or mobile version.

Keywords: *Java, Swing, JDBC, MySQL, Inventory Management, Order Management, PDF Bill, iText.*

TABLE OF CONTENTS

DECLARATION ii
RECOMMENDATIONS iii
CERTIFICATE iv
ACKNOWLEDGEMENTS v
ABSTRACT vi
CHAPTER 1: INTRODUCTION 1
1.1 Introduction 1
1.2 Problem Statement 1
1.3 Objectives 2
1.4 Scope and Limitations 2
1.4.1 Scope 2
1.4.2 Limitations 3
1.5 Development Methodology 3
CHAPTER 2: BACKGROUND STUDY AND LITERATURE REVIEW 5
2.1 Background Study 5
2.1.1 Object-Oriented Programming 5
2.1.2 Java Swing and Event Handling 5
2.1.3 JDBC and the MySQL Database 6
2.1.4 Exception Handling, Regular Expressions and File I/O 6
2.2 Literature Review 6
2.2.1 Review of Related Research / Articles 6
2.2.2 Study of Existing Systems 8
2.2.3 Comparison and Research Gap 8
CHAPTER 3: SYSTEM ANALYSIS 10
3.1 Requirement Analysis 10
3.1.1 Functional Requirements 10
3.1.2 Non-Functional Requirements 11
3.2 Feasibility Study 12
3.2.1 Technical Feasibility 12
3.2.2 Operational Feasibility 12
3.2.3 Economic Feasibility 13
3.2.4 Schedule Feasibility 13
3.3 Analysis 13
3.3.1 Object Modelling 13
3.3.2 Dynamic Modelling 14

3.3.3 Process Modelling....18
CHAPTER 4: SYSTEM DESIGN....19
4.1 Design....19
4.1.1 Refined Class Diagram....19
4.1.2 Database Design....20
4.1.3 Architecture Design....24
4.1.4 Interface Design....24
4.2 Algorithm Details....26
Algorithm 1: User authentication....26
Algorithm 2: Input validation (example: product)....26
Algorithm 3: Add to cart and save order....26
CHAPTER 5: IMPLEMENTATION AND TESTING....29
5.1 Implementation....29
5.1.1 Tools Used....29
5.1.2 Implementation Details of Modules....29
5.2 Testing....33
5.2.1 Unit Testing....33
5.2.2 System / Integration Testing....35
5.3 Result Analysis....36
CHAPTER 6: CONCLUSION AND FUTURE RECOMMENDATIONS....38
6.1 Conclusion....38
6.2 Future Recommendations....38
REFERENCES....39
APPENDICES....40
Appendix A — Screenshots of Major Screens....40
Appendix B — Source Code of Key Classes....44
B.1 dao/ConnectionProvider.java....44
B.2 Login.java – login button handler....44
B.3 ManageUser.java – input validation....45
B.4 ManageOrder.java – add to cart....46
B.5 ManageOrder.java – save order (stock update and order insert)....46
Appendix C — User Manual....48

LIST OF FIGURES

Figure 1.1Waterfall model applied to the Inventory Management System 4
Figure 3.1 Use case diagram of the Inventory Management System *11*
Figure 3.2 Gantt chart of the project schedule.....13
Figure 3.3 Class diagram (analysis level).....14
Figure 3.4 Object diagram for a sample order.....14
Figure 3.5 Sequence diagram for login 15
Figure 3.6 Sequence diagram for adding a product.....15
Figure 3.7 Sequence diagram for placing an order and generating the bill.....16
Figure 3.8 Activity diagram for order processing 17
Figure 3.9 State diagram of a product.....17
Figure 3. 10 Context diagram (DFD Level 0) 18
Figure 3.11 DFD Level 1.....18
Figure 4.1 Refined class diagram..... *19*
Figure 4.2 Package diagram 20
Figure 4.3 ER diagram 20
Figure 4.4 Relational schema.....21
Figure 4.5 System architecture diagram 24
Figure 4.6 Component diagram.....24
Figure 4.7 Interface wireframes of the main screens 25
Figure 4.8 Flowchart of the login algorithm.....27
Figure 4.9 Flowchart of the order processing algorithm.....28
Figure A.1 Login screen *40*
Figure A.2 Home screen (dashboard) 40
Figure A. 3 Manage User screen.....41
Figure A.4 Manage Category screen.....41
Figure A.5 Manage Product screen.....42
Figure A.6 Manage Customer screen 42
Figure A.7 Manage Order screen.....43
Figure A.8 View Order screen.....43

LIST OF TABLES

Table 2.1 Comparison of existing systems and the proposed system 8
Table 3.1 Table of Functional Requirements *10*
Table 3.2 Table of Feasibility Study 12
Table 4.1 Structure of the appuser table *21*
Table 4.2 Structure of the category table 22
Table 4.3 Structure of the product table 22
Table 4.4 Structure of the customer table 22
Table 4.5 Structure of the orderDetail table 23
Table 5.1 Tools used *29*
Table 5.2 Java concepts used *32*
Table 5.3 Unit test cases for input validation *33*
Table 5.4 System / integration test cases *35*

LIST OF ABBREVIATIONS

API – Application Programming Interface

CRUD – Create, Read, Update, Delete

DAO – Data Access Object

DB – Database

DFD – Data Flow Diagram

ER – Entity Relationship

FK – Foreign Key

GUI – Graphical User Interface

IDE – Integrated Development Environment

IMS – Inventory Management System

JDBC – Java Database Connectivity

JDK – Java Development Kit

OOP – Object-Oriented Programming

PDF – Portable Document Format

PK – Primary Key

SQL – Structured Query Language

UML – Unified Modeling Language

CHAPTER 1: INTRODUCTION

1.1 Introduction

The Inventory Management System (IMS) is a desktop-based application that helps a small shop or store keep an accurate record of its products, product categories, customers, system users and sales orders in one place. It is intended for the shop owner (Super Admin) and the staff who operate the system (Admin). Instead of writing stock details in registers or spreadsheets, the user logs in, records products under categories, selects a customer, adds products to a cart, and the system automatically reduces the stock and produces a printable PDF bill for the order.

The system is developed entirely in Java. The graphical user interface is built with Java Swing using the NetBeans GUI builder, data is stored in a MySQL database accessed through JDBC (MySQL Connector/J), and the iText library is used to generate PDF bills. The project follows an object-oriented structure, with the code divided into the user-interface classes, a dao package for database connection and a common package for shared utilities.

1.2 Problem Statement

Many small retail businesses still maintain stock and sales records manually or in loosely organized spreadsheets. This creates several problems that affect the daily operation of the business:

a) **Time consumption:** searching for a product, a customer or an old bill in paper registers takes a long time, and preparing a bill by hand for every sale slows down the counter.

b) **Data redundancy and inconsistency:** the same customer or product details are written repeatedly in different places and easily become inconsistent.

c) **Calculation and entry errors:** manual calculation of sub-totals, totals and remaining stock often leads to mistakes, resulting in wrong bills or incorrect stock counts.

d) **Poor record searching and tracking:** it is difficult to see the complete order history of a customer or to know the current quantity of a product at any moment.

e) **Lack of security and access control:** paper records and shared files can be changed by anyone, and there is no way to separate the rights of the owner from those of staff members.

The proposed system solves these problems by storing all information in a central database, validating every input, deducting stock automatically when an order is saved, generating a bill for every order, and restricting access through a login screen with user roles.

1.3 Objectives

The main objective of this project is to develop a desktop-based inventory and order management system in Java. The specific objectives are:

a) To develop a secure login module that allows only active users to access the system and shows menu options according to the user role.

b) To provide modules for adding, viewing and updating categories, products, customers and users, with validation of names, mobile numbers, e-mail addresses, quantity and price.

c) To maintain the stock of every product and to reduce the stock automatically when an order is placed, while preventing orders that exceed the available quantity.

d) To create an order module with a shopping cart that calculates sub-totals and the total amount, saves the order against a customer and generates a PDF bill automatically.

1.4 Scope and Limitations

1.4.1 Scope

The system covers the core stock and sales activities of a small business. It includes seven functional modules: Login and Logout, User Management, Category Management, Product Management, Customer Management, Order Management (cart

and billing) and View Order. The target users are the shop owner (Super Admin) and the shop staff (Admin). The boundary of the system is a single computer running the Java application and a MySQL server; suppliers, purchase orders, payments and accounting are outside the boundary.

1.4.2 Limitations

a) The system is a standalone desktop application connected to a MySQL database on the local machine (localhost); it is not a web or multi-branch system.

b) It does not support suppliers, purchase orders, returns, discounts, tax calculation or online payment.

c) Passwords are stored as plain text in the database and the database connection details are written inside the source code, so the system is suitable for learning and small local use only.

d) There is no delete option for records and no search or filter box on the tables; records are selected by clicking a table row.

e) PDF bills are saved in a fixed folder (the D:\ drive) and are opened using a Windows command, so the bill feature works on Windows systems with that drive available.

1.5 Development Methodology

The Incremental model was followed for this project. It was selected because the inventory system can be divided into independent modules (login, products, customers, orders and billing), and a working core could be delivered early and extended in stages. The owner could use the first version while later modules were still being built, and any change in requirements could be handled in a later increment without redoing completed work. The phases were applied as follows:

a) Requirement gathering: the problems of manual record keeping were identified, and the functions needed by the owner and staff were listed and prioritised. The functions were then grouped into four increments: Increment 1 (login and product management), Increment 2 (customer management), Increment 3 (order processing and bill generation) and Increment 4 (reports and improvements).

b) System analysis: the overall system was modelled with a use case diagram and a class diagram. For each increment, the related sequence, activity and state diagrams and data flow diagrams were prepared just before that increment was built.

c) System design: the overall architecture, ER diagram and relational schema were defined at the start so that every increment would fit together. The detailed class design and screen wireframes were then refined separately for each increment.

d) Implementation: each increment was coded in turn. The Swing screens were created in NetBeans, the MySQL tables were added as each module required them, and the logic of the module was coded in Java. Each new increment was integrated with the previously completed ones.

e) Testing: every increment was tested on its own (input validation and module functionality). It was then tested together with earlier increments (integration testing) to confirm that nothing already working had broken. After the final increment, the complete flow from login to bill generation was tested as a whole.

f) Deployment and maintenance: each tested increment was released for use on the user's computer with the MySQL database. Errors found during testing and use were corrected in the next increment, and further improvements are listed for the future.

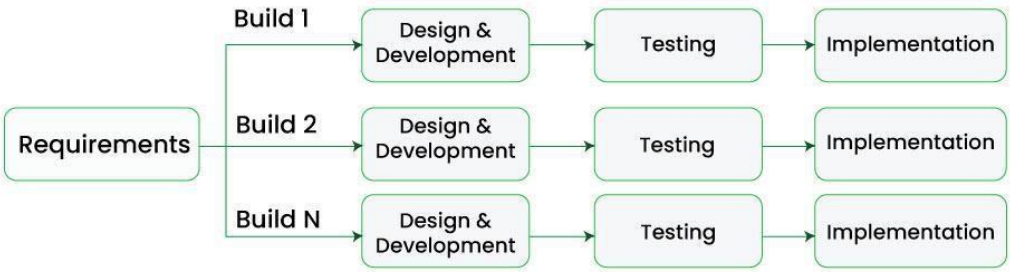


Figure 1.1Waterfall model applied to the Inventory Management System

CHAPTER 2: BACKGROUND STUDY AND LITERATURE REVIEW

2.1 Background Study

The proposed Inventory Management System is a Java-based desktop application developed to manage products, product categories, customers, users, and sales orders. The system provides a graphical interface through which authorized users can interact with the inventory database. Instead of maintaining inventory records manually, the system stores information in a structured MySQL database and allows users to add, update, view, and manage records through different management modules.

The application is developed using Java and Java Swing in Apache NetBeans. Java Swing is used to create the graphical user interface, while JDBC (Java Database Connectivity) is used to communicate with the MySQL database. The project uses the MySQL Connector/J 8.0.20 JDBC driver for database connectivity. The application contains separate JFrame-based interfaces such as Login, Home, ManageUser, Manage Category, ManageProduct, ManageCustomer, and ManageOrder. Database connectivity is centralized through the ConnectionProvider class.

2.1.1 Object-Oriented Programming

Java is a platform-independent, object-oriented language in which a program is organized as classes and objects [1], [2]. In this project every screen is a class (for example Login, ManageProduct, ManageOrder) and each class keeps its own data as private attributes such as productPk or finalTotalPrice. Inheritance is used because every screen class extends javax.swing.JFrame. Constructor overloading is used in the Home class, which has a default constructor and another constructor Home (String role) that receives the role of the logged-in user. Encapsulation is applied by declaring fields and event handlers private and keeping the database access in a separate class. Packages (dao, common, inventory. management.system) group related classes.

2.1.2 Java Swing and Event Handling

Swing is the Java toolkit for building desktop graphical user interfaces. The project uses components such as JFrame, JLabel, JTextField, JPasswordField, JButton, JComboBox, JTable and JOptionPane. User actions such as clicking a button or a table

row are handled through the event delegation model: event handlers like btnSaveActionPerformed and tableProductMouseClicked are registered on the components and run when the event occurs. The tables use DefaultTableModel to add rows dynamically.

2.1.3 JDBC and the MySQL Database

Java Database Connectivity (JDBC) is the standard API that lets a Java program run SQL statements on a relational database. The project loads the driver class com.mysql.cj.jdbc.Driver from MySQL Connector/J 8.0.20 (a Type-4, pure-Java driver) and obtains a Connection through DriverManager. Data is inserted and updated using PreparedStatement with ? placeholders, which is safer and faster than building SQL by string concatenation, while simple queries use Statement and ResultSet.

2.1.4 Exception Handling, Regular Expressions and File I/O

Every database operation is placed inside a try ... catch block and any exception is shown to the user through a JOptionPane message. Input validation is performed with regular expressions and the String.matches() method. For file output, the order module writes the bill using FileOutputStream together with the iText library, which creates PDF documents, tables and cells.

2.2 Literature Review

Seven research papers and articles and two existing software systems were studied. The summary of each source and its relation to this project is given below.

2.2.1 Review of Related Research / Articles

Axsäter explains the importance of inventory control in managing the flow of materials and reducing costs. The book discusses inventory models, forecasting, replenishment, and different approaches for controlling stock. It emphasizes that an effective inventory control system requires an understanding of demand and inventory levels. These concepts are relevant to an Inventory Management System because the system can help businesses monitor available stock and maintain appropriate inventory levels. [1]

Toomey's work presents fundamental principles and techniques used in inventory management. It covers inventory, distribution, production, scheduling, and supply-chain-related concepts. The literature provides a foundation for understanding how

inventory should be organized and controlled within an organization. For an inventory management software project, these principles are useful for designing functions such as product records, stock monitoring, and inventory control. [2]

Mapes describes inventory control systems as systems designed to maintain inventory at an appropriate level while ensuring continuity of supply. The literature discusses continuous-review and periodic-review inventory systems. This supports the development of an Inventory Management System because computerized systems can provide a structured way to monitor inventory and maintain accurate stock information. [3]

Axsäter's third edition expands inventory-control concepts including forecasting, lot sizing, safety stock, reorder points, KANBAN policies, and Material Requirements Planning (MRP). The book also focuses on the practical application of inventory-control methods in real systems. These concepts are particularly relevant to computerized inventory systems because software can store inventory information and assist users in monitoring stock levels and making replenishment decisions. [4]

Inventory management involves maintaining the appropriate quantity of products so that customer demand can be met without unnecessarily increasing storage and purchasing costs. The study of inventory management emphasizes the importance of accurate stock records, timely replenishment, and effective monitoring of inventory levels. These concepts are relevant to the proposed Inventory Management System because the system provides a computerized method for recording products, monitoring quantities, managing sales, and maintaining customer and order information. By replacing manual record keeping with a computerized database, the system can improve the accuracy and accessibility of inventory information. [5]

2.2.2 Study of Existing Systems

Zoho Inventory is a cloud-based inventory application. It offers real-time stock tracking, barcode and RFID based tracking, low-stock alerts with reorder points, multi-warehouse management and selling through multiple online channels, with a free plan and paid plans. It is powerful but is subscription-based, needs an Internet connection and is more complex than a small local shop requires. [6]

Odoo Inventory is an integrated warehouse application that supports barcode operations, minimum/maximum reordering rules, multi-warehouse routes, lots and serial numbers and links with the sales and purchase apps. It is suitable for large or growing businesses, but its setup and configuration are complicated for a small shop without a technical person. [7]

Spreadsheet or register based record keeping is the most common local method. It is free and familiar, but, as observed in [8], it is labor-intensive, error-prone and gives no access control, no automatic stock deduction and no automatic bill generation.

2.2.3 Comparison and Research Gap

Table 2.1 Comparison of existing systems and the proposed system

Feature	Spreadsheet / Register	Zoho Inventory [10]	Odoo Inventory [11]	Proposed IMS
Stock tracking	Manual	Yes	Yes	Yes (automatic deduction)
Customer-wise order history	Manual	Yes	Yes	Yes
PDF bill generation	No	Yes	Yes	Yes (iText)
User roles / login	No	Yes	Yes	Yes (Super Admin, Admin)

Feature	Spreadsheet / Register	Zoho Inventory [10]	Odoo Inventory [11]	Proposed IMS
Works offline on one PC	Yes	No (cloud)	Partly	Yes
Cost	Free	Subscription (free tier)	License/ subscription	Free (open-source tools)
Ease of setup for a small shop	Easy	Medium	Complex	Easy
Barcode / RFID, multi-warehouse	No	Yes	Yes	No

From the study it is clear that large commercial systems are feature-rich but costly and complex, while small shops still depend on manual methods. Academic systems [3]–[7] are mostly web-based or use other technologies, and some require frameworks that are too heavy for beginners. The research gap is therefore a simple, free, offline, Java desktop system that combines category, product, customer and order management with automatic stock deduction and PDF billing on one computer. The proposed Inventory Management System addresses this gap.

CHAPTER 3: SYSTEM ANALYSIS

3.1 Requirement Analysis

3.1.1 Functional Requirements

The functional requirements are listed module-wise below and are represented by the use case diagram in Figure 3.1.

Table 3.1 Table of Functional Requirements

Module	Functional requirement
Login / Logout	Authenticate a user by e-mail and password; allow only users whose status is Active; validate e-mail format; confirm before logout or exit.
User Management	Add a new Admin user, view all admin users, update user details and change status (Active / Inactive). Only the Super Admin can open this module.
Category Management	Add a category, view all categories in a table and update a category name.
Product Management	Add a product with name, quantity, price, description and category; view products with category name; update product details and add extra quantity to existing stock.
Customer Management	Add, view and update customers with name, mobile number and e-mail.
Order Management	Select a customer and a product, enter quantity, check stock, add to cart, remove from cart, calculate total, save the order, reduce stock and generate a PDF bill.
View Order	Select a customer to list all of his/her orders and click an order to open its PDF bill.

Actors. The Super Admin is the owner and can use all modules including User Management. The Admin is a staff user who can use all modules except User Management. Use cases**.** Login, Logout, Manage Users, Manage Categories, Manage Products, Manage Customers, Place Order (with the included use case Generate PDF Bill) and View Orders.

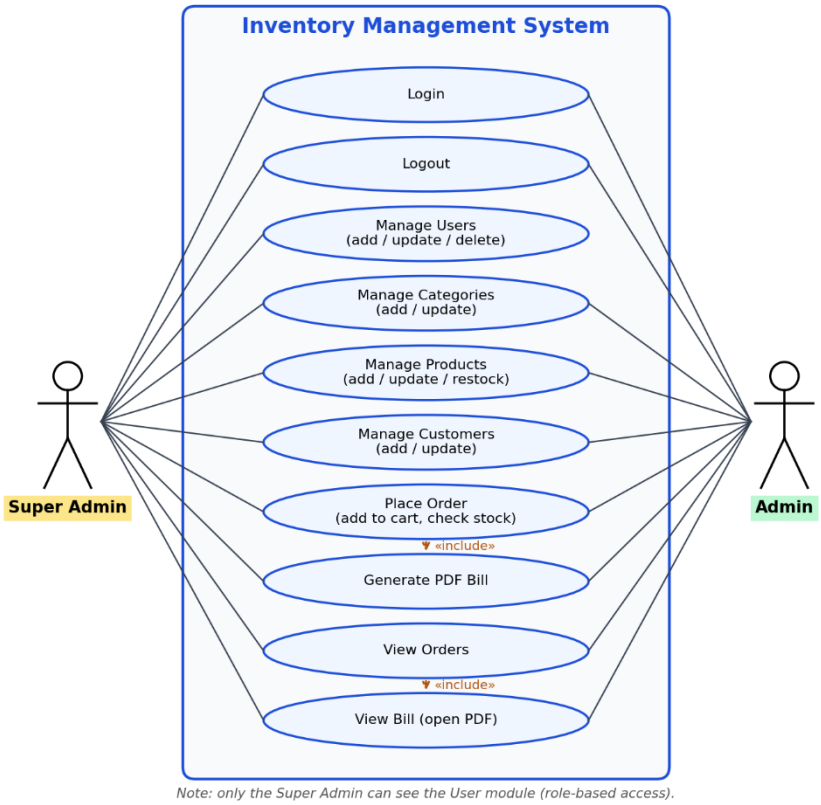


Figure 3.1 Use case diagram of the Inventory Management System

3.1.2 Non-Functional Requirements

a) **Performance:** every screen should load its table data and respond to a click within a few seconds on an ordinary computer.

b) **Usability:** simple screens with labelled fields, buttons (Save, Update, Reset, Close) and clear confirmation or error messages through dialog boxes.

c) **Security:** access is allowed only after login; only Active users can log in; input is validated; User Management is restricted to the Super Admin.

d) **Reliability:** exceptions are caught and reported instead of terminating the program; stock cannot become negative because orders above the available quantity are rejected.

e) **Maintainability:** code is divided into screen classes and dao / common packages and uses PreparedStatement for insert and update operations.

f) **Portability:** being written in Java, the application runs on any operating system that has a JDK and MySQL (the bill-opening feature is currently Windows-specific).

3.2 Feasibility Study

3.2.1 Technical Feasibility

The system needs only ordinary hardware and freely available software, all of which were available to the team.

Table 3.2 Table of Feasibility Study

Item	Requirement
Hardware	A computer with at least 4 GB RAM, dual-core processor and 500 MB free disk space
Operating system	Windows 10 / 11 (Linux and macOS can run the application except the bill-opening command)
Java	JDK (the NetBeans project is configured for Java source level 26)
IDE	Apache NetBeans (with the Swing GUI builder)
Database	MySQL Server (database name: inventory)
Libraries	MySQL Connector/J 8.0.20 and the iText PDF library

3.2.2 Operational Feasibility

The screens are simple and each module follows the same pattern of a table on the left and a form on the right, so a shop assistant with basic computer knowledge can learn the system quickly. It follows the shop's natural workflow of adding products, selecting a customer, preparing a bill and viewing old bills, and it replaces manual registers without changing the way the business works.

3.2.3 Economic Feasibility

The development cost is very low because only free and open-source tools (Java, NetBeans, MySQL, Connector/J) and a freely available PDF library were used and the work was done on the team's own computers. The benefits are saved time in billing and searching, fewer calculation mistakes and better stock control. Compared with subscription software such as [10], the proposed system has no recurring license cost.

3.2.4 Schedule Feasibility

The project was planned for one semester (about twelve weeks), with requirement gathering, analysis, design, coding, testing and documentation overlapping as shown in Figure 3.2. The planned schedule is realistic because the modules are small and similar to each other, so the project can be completed within the semester.



Figure 3.2 Gantt chart of the project schedule

3.3 Analysis

The system is modelled using the object-oriented approach.

3.3.1 Object Modelling

Five main entity classes were identified from the requirements: AppUser, Category, Product, Customer and OrderDetail. A **Category** contains many **Products** (one-to-many), a **Customer** places many **Orders** (one-to-many) and an order contains many products through the cart (many-to-many at analysis level). The analysis-level class diagram is given in Figure 3.3 and a sample object diagram in Figure 3.4.

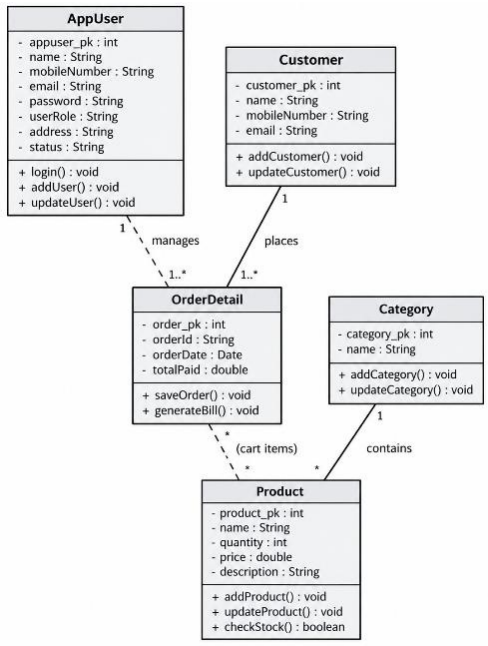


Figure 3.3 Class diagram (analysis level)

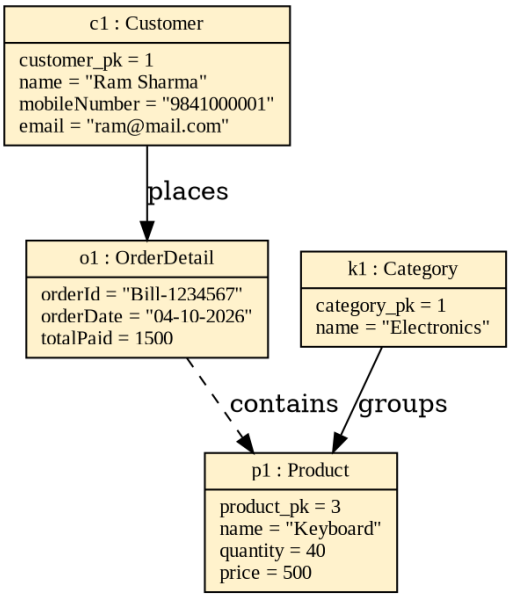


Figure 3.4 Object diagram for a sample order

3.3.2 Dynamic Modelling

The interaction of objects over time is shown with sequence diagrams for three key operations. Figure 3.5 shows the login process: the Login form validates the e-mail format, obtains a connection from ConnectionProvider, queries the appuser table and opens Home with the user's role. Figure 3.6 shows how a product is added, and Figure 3.7 shows how an order is saved and the PDF bill opened.

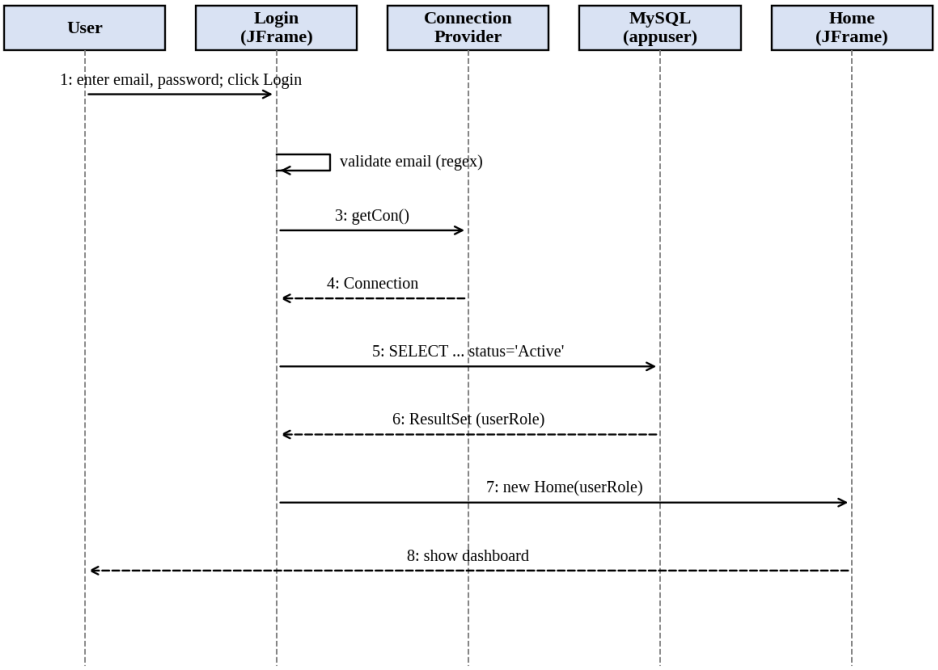


Figure 3.5 Sequence diagram for login

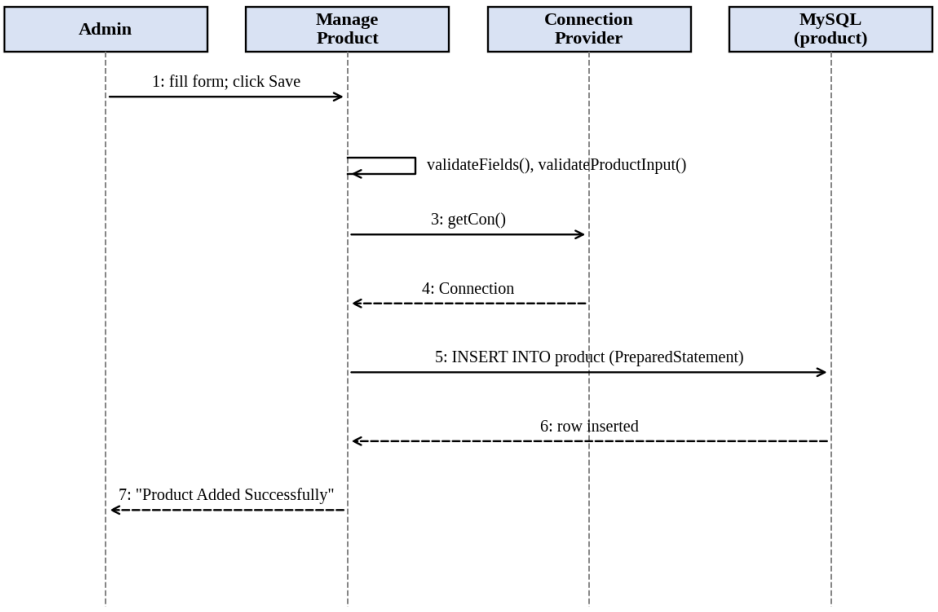


Figure 3.6 Sequence diagram for adding a product

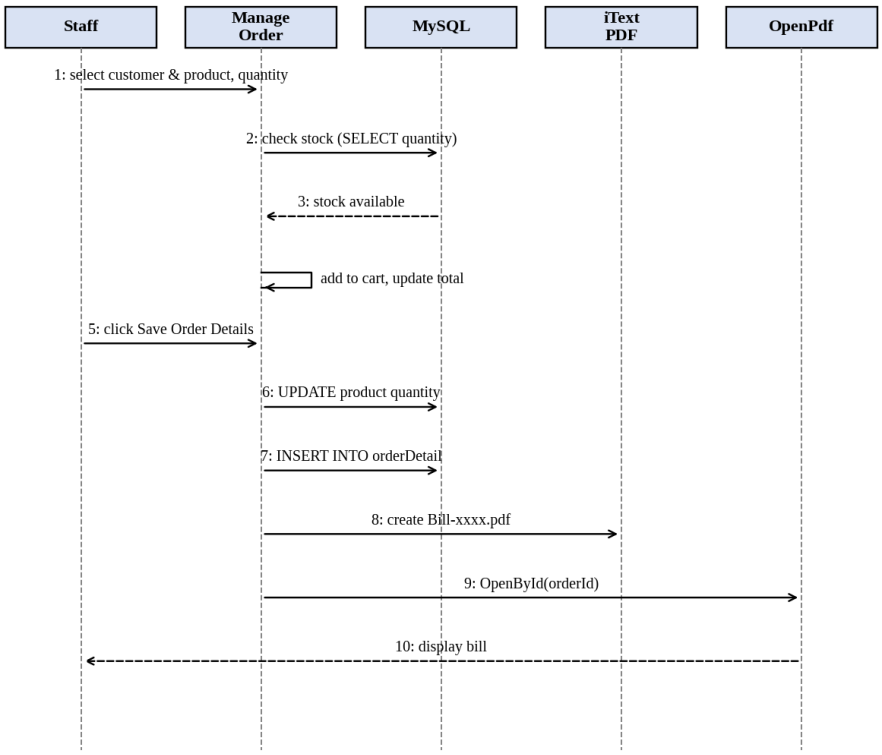


Figure 3.7 Sequence diagram for placing an order and generating the bill

The activity diagram in Figure 3.8 describes the flow of the order process, including the decisions for stock availability, duplicate products in the cart and an empty cart. The state diagram in Figure 3.9 shows the states of a product: it is In Stock while its quantity is above zero and becomes Out of Stock when orders reduce the quantity to zero, and returns to *In Stock* when quantity is added.

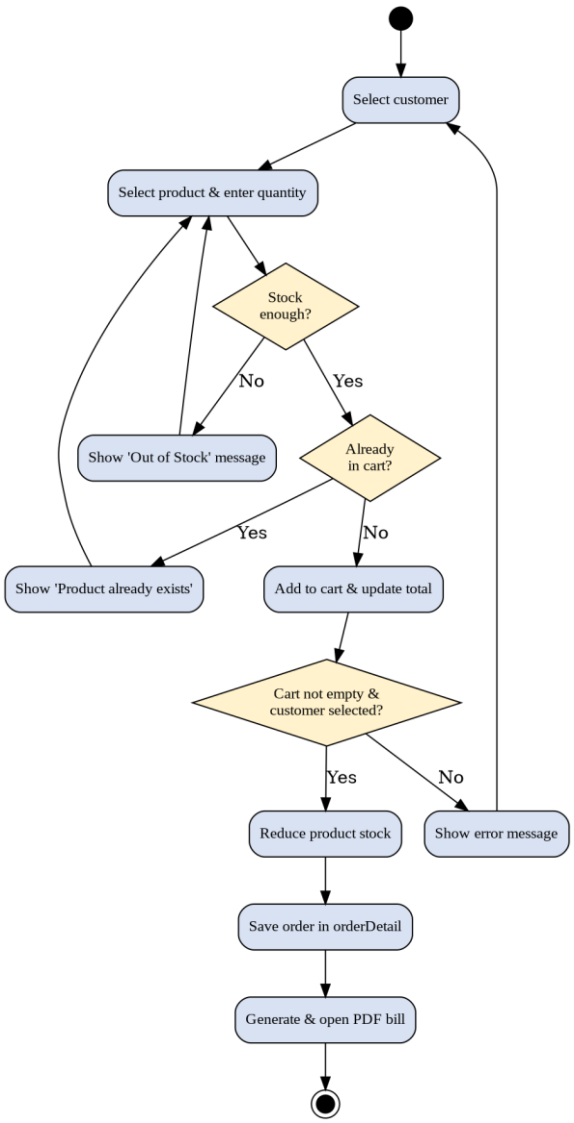


Figure 3.8 Activity diagram for order processing

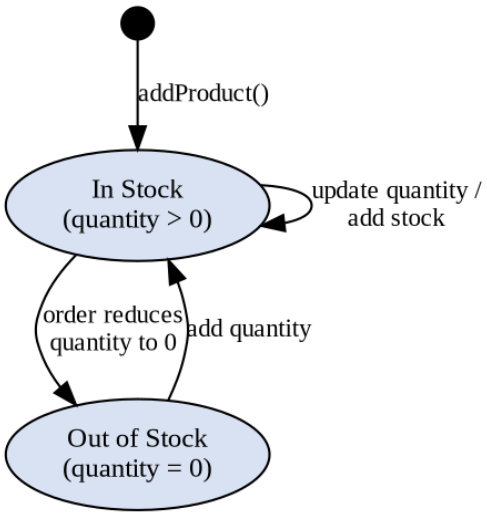


Figure 3.9 State diagram of a product

3.3.3 Process Modelling

Figure 3.10 is the context diagram (DFD Level 0) in which the Admin or Super Admin is the only external entity exchanging data with the system. Figure 3.11 is the DFD Level 1 which divides the system into four processes: Authenticate User, Manage Master Data, Process Order and View Orders, and shows the data stores appuser, category / product / customer and orderDetail.

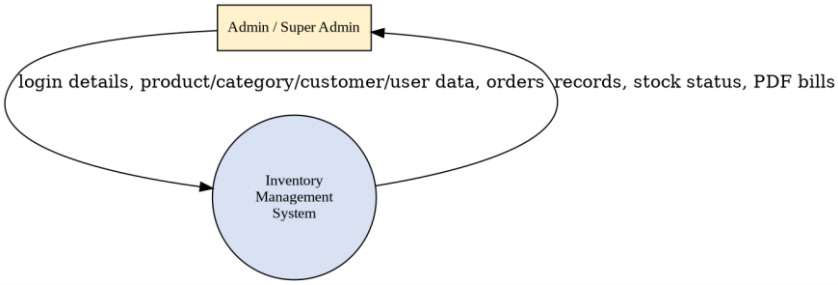


Figure 3. 10 Context diagram (DFD Level 0)

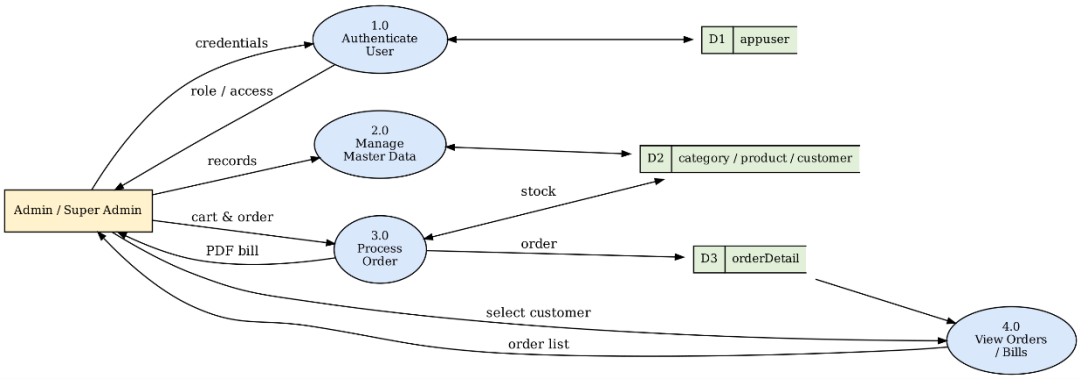


Figure 3.11 DFD Level 1

CHAPTER 4: SYSTEM DESIGN

4.1 Design

4.1.1 Refined Class Diagram

The refined class diagram (Figure 4.1) shows the classes exactly as implemented in code, with data types, access modifiers (+ public, − private) and relationships. All eight screen classes inherit from javax.swing.JFrame. Login creates Home, and Home opens the six management screens. The screen classes depend on dao.ConnectionProvider for the database connection; common.OpenPdf depends on dao.InventoryUtils for the bill folder path. The NetBeans generated members (initComponents() and the Swing component fields) are omitted for clarity. Since the project has no separate entity classes, the entities of the analysis model are represented by database tables and table rows.

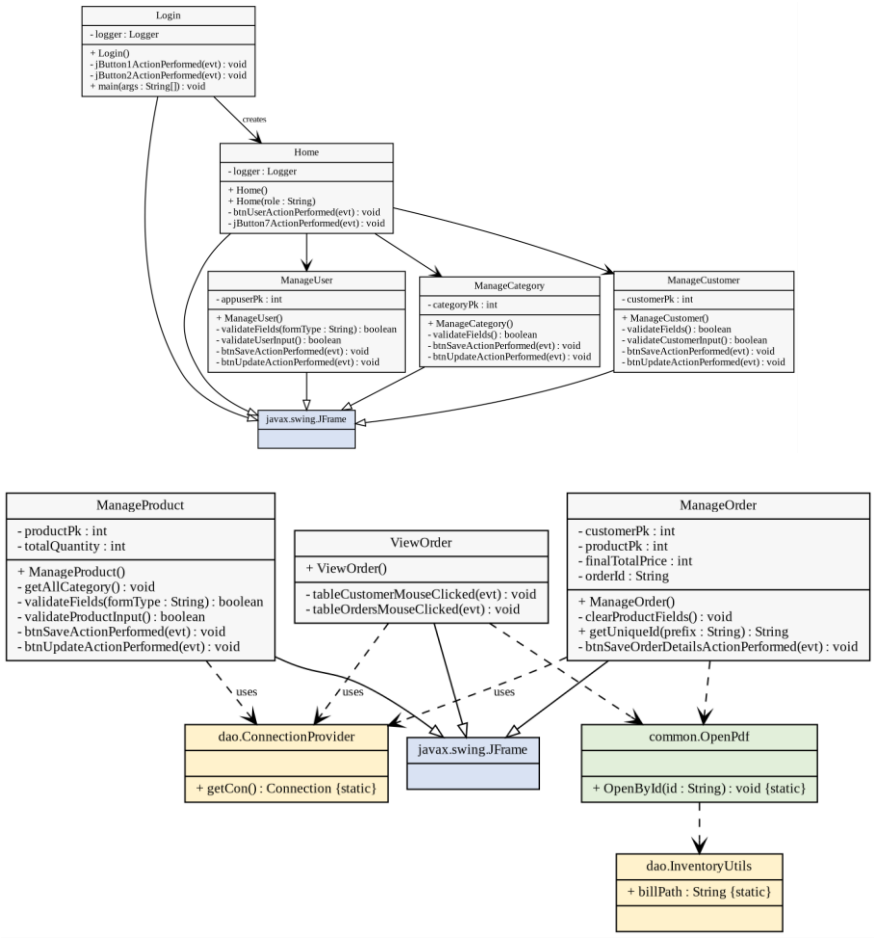


Figure 4.1 Refined class diagram

The classes are grouped into four packages, as shown in Figure 4.2. The default package contains the screen classes, dao contains the connection and utility classes, common contains OpenPdf, and inventory.management.system holds the generated main class.

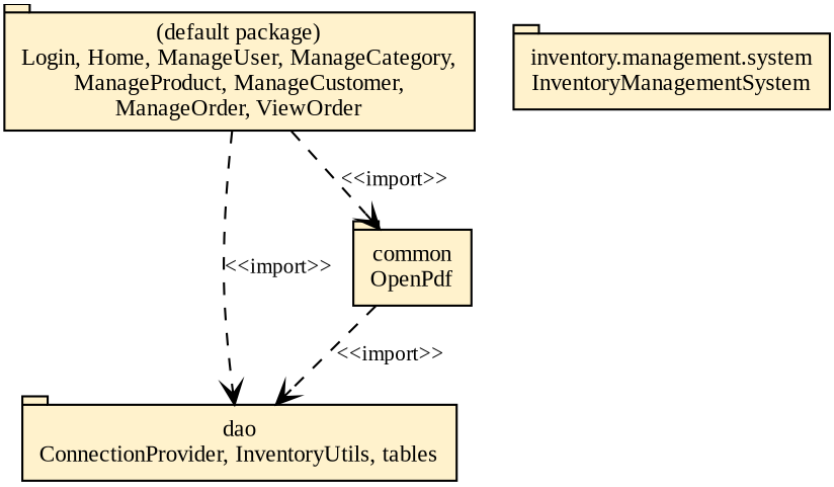


Figure 4.2 Package diagram

4.1.2 Database Design

The database named inventory has five tables. The ER diagram (Figure 4.3) shows that one category contains many products and one customer places many orders. The appuser table is independent and stores the system users with their role and status.

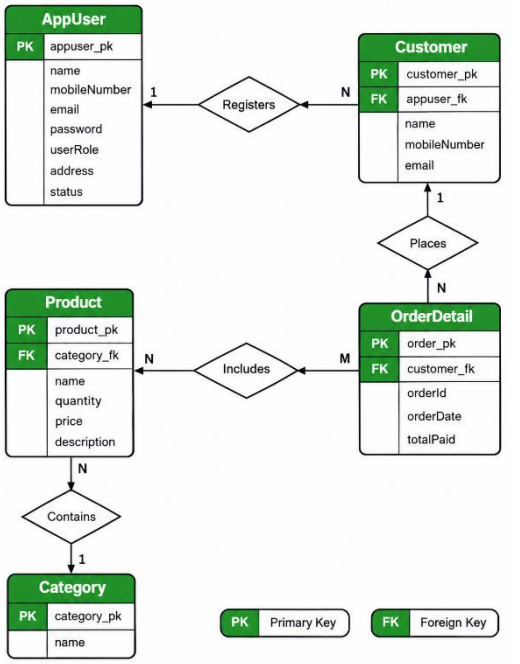


Figure 4.3 ER diagram

The relational schema is shown in Figure 4.4. The foreign keys product.category_fk and orderDetail.customer_fk refer to category_pk and customer_pk respectively. In the table-creation statements used in the project these relationships are maintained by the application and are not declared as FOREIGN KEY constraints.

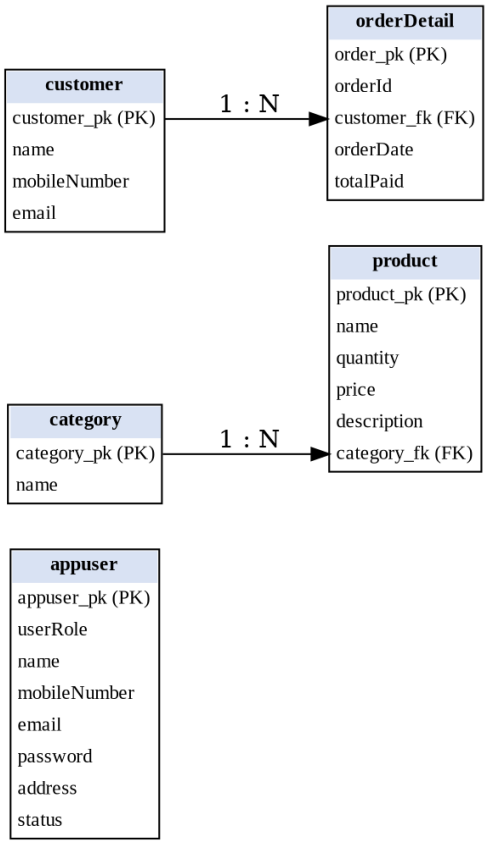


Figure 4.4 Relational schema

The structure of each table is given in Tables 4.1 to 4.5.

Table 4.1 Structure of the appuser table

Field name	Data type	Size	Constraint	Description
appuser_pk	INT	–	PRIMARY KEY, AUTO_INCREMENT	Unique user id
userRole	VARCHAR	50	–	SuperAdmin or Admin
name	VARCHAR	200	–	Full name

Field name	Data type	Size	Constraint	Description
mobileNumber	VARCHAR	50	–	Mobile number
email	VARCHAR	200	–	Login e-mail
password	VARCHAR	50	–	Login password
address	VARCHAR	200	–	Address
status	VARCHAR	50	–	Active or Inactive

Table 4.2 Structure of the category table

Field name	Data type	Size	Constraint	Description
category_pk	INT	–	PRIMARY KEY, AUTO_INCREMENT	Unique category id
name	VARCHAR	200	–	Category name

Table 4.3 Structure of the product table

Field name	Data type	Size	Constraint	Description
product_pk	INT	–	PRIMARY KEY, AUTO_INCREMENT	Unique product id
name	VARCHAR	200	–	Product name
quantity	INT	–	–	Stock available
price	INT	–	–	Price per unit
description	VARCHAR	500	–	Product description
category_fk	INT	–	Refers to category.category_pk	Category of the product

Table 4.4 Structure of the customer table

Field name	Data type	Size	Constraint	Description
customer_pk	INT	–	PRIMARY KEY, AUTO_INCREMENT	Unique customer id
name	VARCHAR	200	–	Customer name
mobileNumber	VARCHAR	50	–	Mobile number
email	VARCHAR	200	–	E-mail address

Table 4.5 Structure of the orderDetail table

Field name	Data type	Size	Constraint	Description
order_pk	INT	–	PRIMARY KEY, AUTO_INCREMENT	Unique order id
orderId	VARCHAR	200	–	Bill number, e.g. Bill-123456
customer_fk	INT	–	Refers to customer.customer_pk	Customer who placed the order
orderDate	VARCHAR	200	–	Date in dd-MM-yyyy format
totalPaid	INT	–	–	Total amount of the order

Normalization. All tables are in First Normal Form because every column holds a single atomic value and each table has a primary key. They are in Second Normal Form because every non-key attribute depends on the whole of a single-column primary key. They are in Third Normal Form because there are no transitive dependencies: the category name is stored only in category and the customer details only in customer, while product and orderDetail keep only the foreign keys. The items of an order are kept only in the printed bill and are not stored in a separate order-item table.

4.1.3 Architecture Design

The system follows a **2-tier (Swing + JDBC + DB)** architecture. The Java Swing screens and the Java classes that process data run together on the user's computer (the client tier), and they communicate through JDBC with the MySQL server (the data tier), which in this project runs on the same computer at localhost:3306. The iText library creates the PDF bills, which are stored in the file system. The architecture is shown in Figure 4.5 and the components in Figure 4.6.

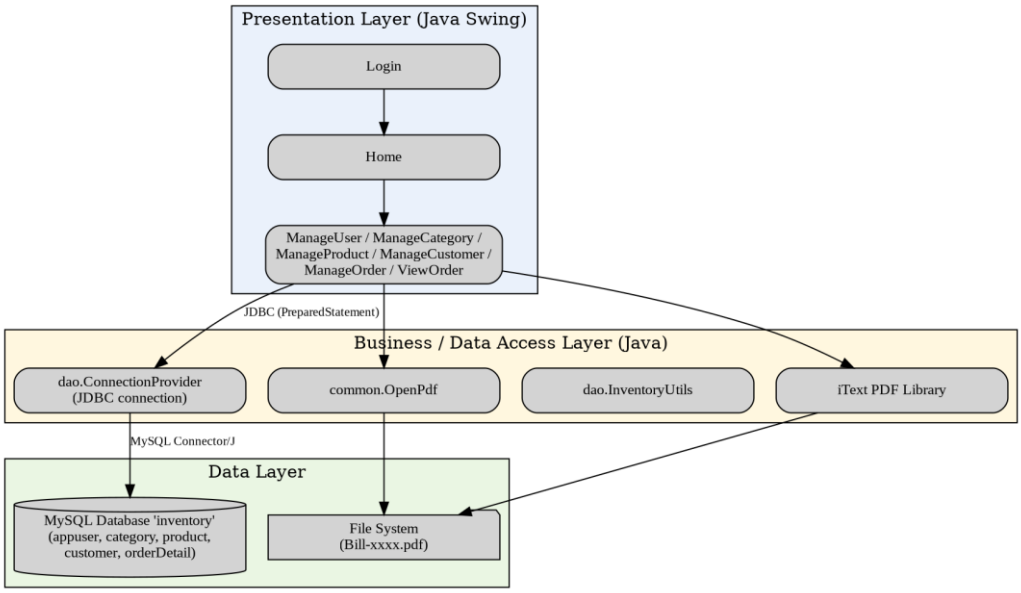


Figure 4.5 System architecture diagram

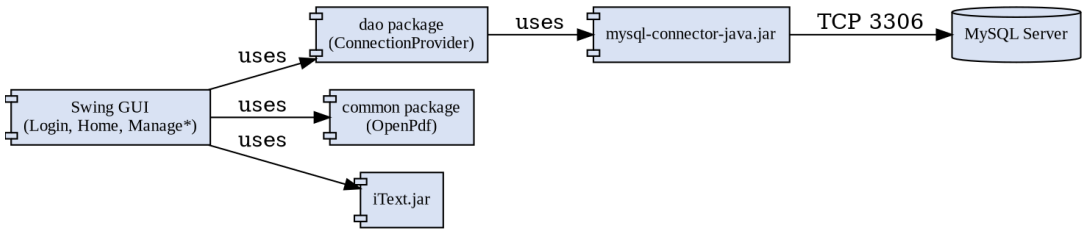


Figure 4.6 Component diagram

A deployment diagram is not applicable because the system is not a Socket, RMI or Servlet/JSP application; all components run on one machine.

4.1.4 Interface Design

The main screens were designed as wireframes before coding (Figure 4.7). The Login screen has e-mail and password fields with Login and Close buttons. The Home screen shows a menu bar of buttons (User, Category, Product, Customer, Order, View Order,

Logout). Each management screen has a table on the left that lists the records and a form on the right with Save, Update, Reset and Close buttons. The Order screen has three tables (customers, products and cart), the selected customer and product details, the total amount and the order buttons. The View Order screen has a customer table and an order table.

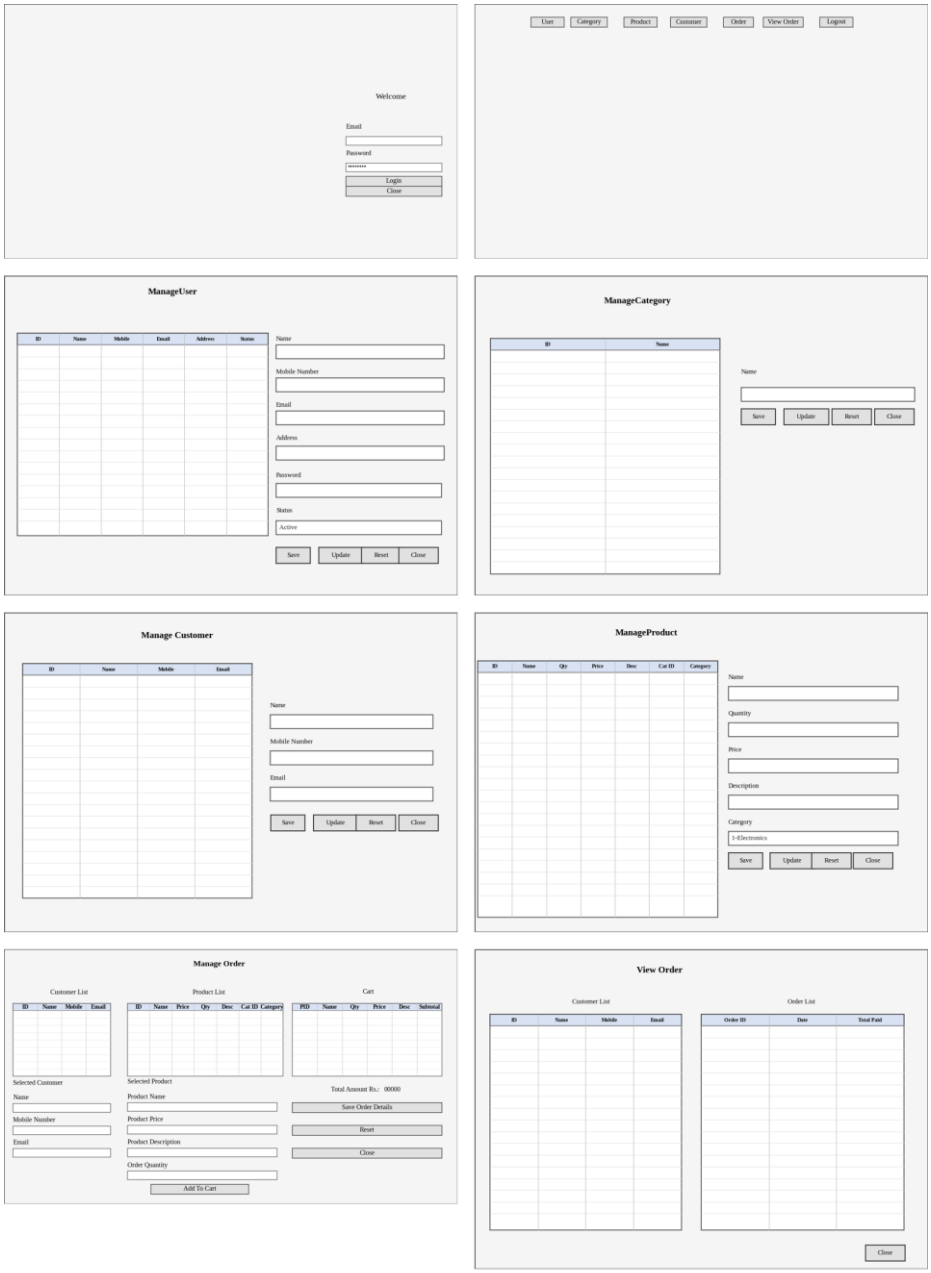


Figure 4.7 Interface wireframes of the main screens

4.2 Algorithm Details

Algorithm 1: User authentication

```
Step 1. Read email and password from the login form.
Step 2. If email does not match ^[A-Za-z0-9+_.-]+@[A-Za-z0-9.-]+$ then
    show "Please enter a valid email address" and stop.
Step 3. Get a database connection.
Step 4. Run: SELECT * FROM appuser
      WHERE email = ? AND password = ? AND status = 'Active'
Step 5. If a record is found then
    read userRole, hide the login form and open Home(userRole).
  Else
    show "Incorrect Email or Password".
```

Algorithm 2: Input validation (example: product)

```
Step 1. If name, price, description (and quantity for a new product) is empty
    show "All fields are required" and stop.
Step 2. If name does not match [a-zA-Z0-9 ]+ then show message and stop.
Step 3. If quantity does not match [0-9]+ then show message and stop.
Step 4. If price does not match [0-9]+(\.[0-9]+)? then show message and stop.
Step 5. Accept the input and execute the INSERT / UPDATE statement.
```

Algorithm 3: Add to cart and save order

```
ADD TO CART
Step 1. subTotal = orderQuantity * productPrice
Step 2. Read stock of the product from the database.
Step 3. If orderQuantity > stock then show "Product is out of Stock" and stop.
Step 4. If the product is already in the cart then show message and stop.
Step 5. Add row (productId, name, quantity, price, description, subTotal) to cart.
Step 6. finalTotalPrice = finalTotalPrice + subTotal

SAVE ORDER
Step 1. If cart is empty or no customer is selected then show message and stop.
Step 2. orderId = "Bill-" + System.nanoTime()
```

Step 3. For each cart row: UPDATE product SET quantity = quantity - rowQuantity

Step 4. INSERT INTO orderDetail(orderId, customer_fk, orderDate, totalPaid)

Step 5. Create PDF "orderId.pdf" with header, order details and a table of items.

Step 6. Open the PDF and reset the order screen.

The flowcharts of the login and order algorithms are given in Figures 4.8 and 4.9.

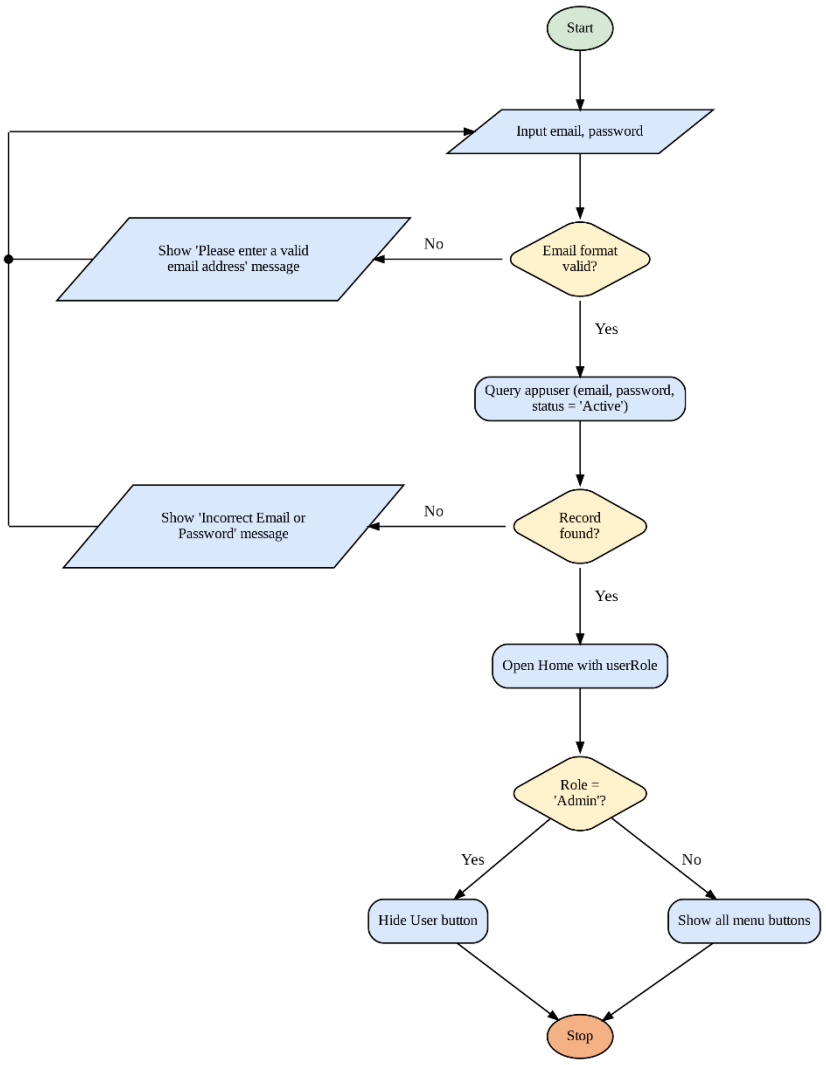


Figure 4.8 Flowchart of the login algorithm

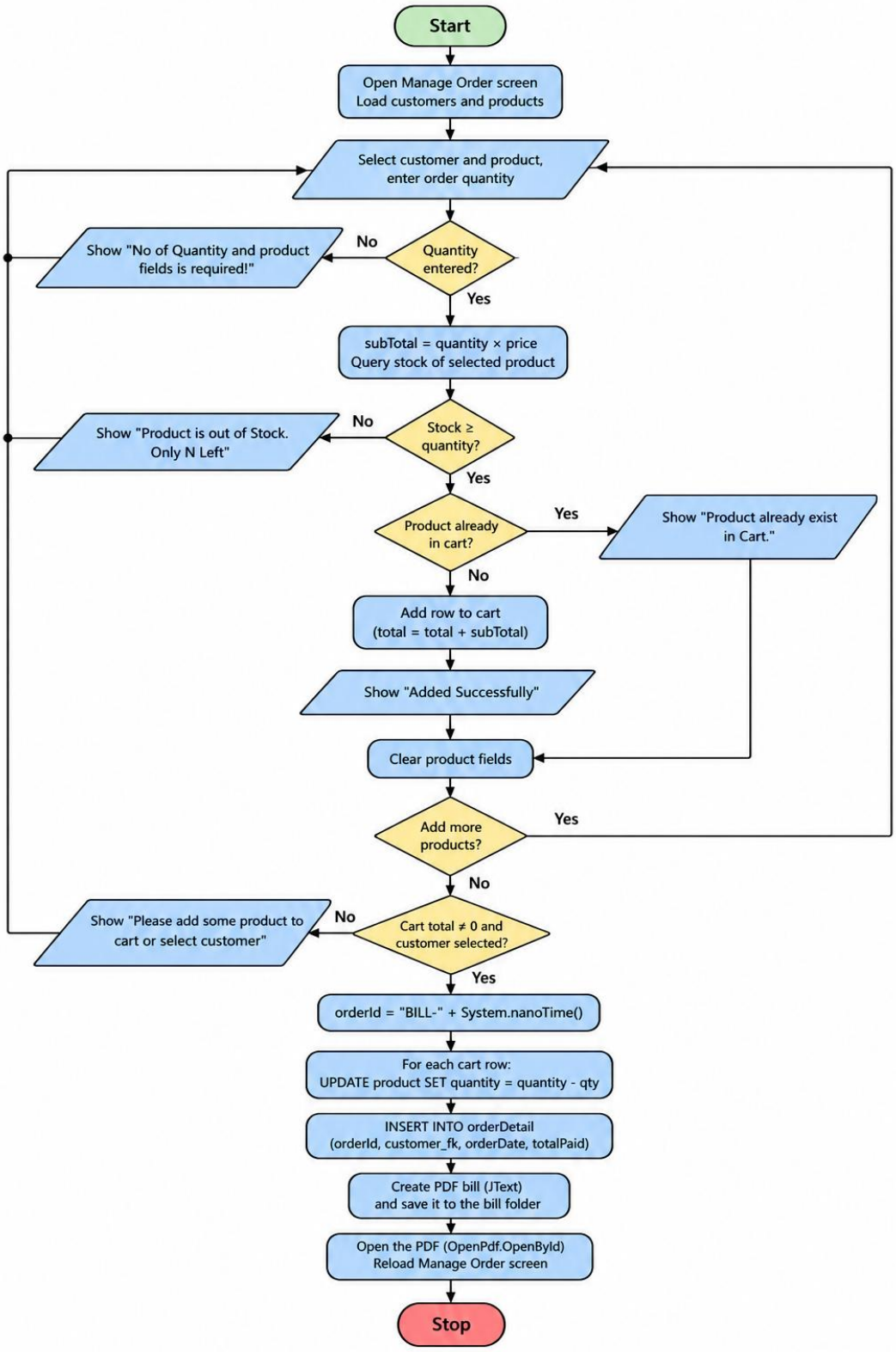


Figure 4.9 Flowchart of the order processing algorithm

CHAPTER 5: IMPLEMENTATION AND TESTING

5.1 Implementation

5.1.1 Tools Used

Table 5.1 Tools used

S.N.	Tool	Purpose
1	Java (JDK)	Programming language used to develop the whole application
2	Apache NetBeans IDE	Writing code and designing Swing forms with the GUI builder
3	MySQL Server	Relational database that stores users, categories, products, customers and orders
4	MySQL Connector/J 8.0.20	JDBC driver (com.mysql.cj.jdbc.Driver) to connect Java with MySQL
5	iText library	Generating the PDF bill
6	Graphviz / drawing tools	Preparing UML, ER, DFD and flow diagrams
7	Git / GitHub	Sharing and version control of the source code within Team7

5.1.2 Implementation Details of Modules

Database connection module (`dao.ConnectionProvider`). This class contains a static method getCon() that loads the MySQL driver and returns a Connection object to the database inventory on localhost:3306. Every screen calls this method, which avoids repeating the connection code. The class uses exception handling: if the connection fails it prints the error and returns null. The helper class InventoryUtils stores the bill folder path and tables is a small utility that was used once to create the tables.

```
public static Connection getCon(){
   try{
      Class.forName("com.mysql.cj.jdbc.Driver");
      Connection con = DriverManager.getConnection(
```

```
            "jdbc:mysql://localhost:3306/inventory?useSSL=false","root","********");
        return con;
    } catch(Exception e){ System.out.println(e); return null; }
}
```

Login and Home modules (`Login`, `Home`). Login extends JFrame. When the Login button is clicked, the e-mail is validated with a regular expression, the appuser table is queried for an active user and Home is opened with the role through the overloaded constructor Home(String role). Home is the dashboard; its buttons create objects of the other screens (new ManageProduct().setVisible(true)). The logout button asks for confirmation using JOptionPane.showConfirmDialog.

```
ResultSet rs = st.executeQuery("select * from appuser where email='" + email
        + "' and password='" + password + "' and status='Active'");
while (rs.next()) {
    temp = 1; setVisible(false);
    new Home(rs.getString("userRole")).setVisible(true);
}
if (temp == 0) JOptionPane.showMessageDialog(null, "Incorrect Email or
Password");
```

User, Category and Customer modules (`ManageUser`, `ManageCategory`, `ManageCustomer`). Each class loads the existing records into a JTable in the formComponentShown event using a DefaultTableModel. Clicking a table row (a MouseEvent) fills the form and enables the Update button, while Save inserts a new record using PreparedStatement. Validation methods such as validateUserInput() use regular expressions to check that the name has only letters, the mobile number starts with 97 or 98 and has 10 digits, and the e-mail is valid.

```
private boolean validateCustomerInput() {
    String mobileNumber = txtMobileNumber.getText().trim();
    if (!mobileNumber.matches("^(97|98)[0-9]{8}$")) {
        JOptionPane.showMessageDialog(null,
            "Mobile number must start with 97 or 98 and contain exactly 10 digits.");
        txtMobileNumber.requestFocus();
        return false;
```

```
    }
    return true;
}
```

Product module (`ManageProduct`). The categories are loaded into a JComboBox as "id-name" text, and the category id is extracted using split("-") when the product is saved. The table is filled using an SQL **inner join** of product and category. In update mode the quantity field is used to add extra stock to the existing quantity.

```
PreparedStatement ps = con.prepareStatement(
  "insert into product(name,quantity,price,description,category_fk)
values(?,?,?,?,?)");
ps.setString(1, name);  ps.setString(2, quantity);  ps.setString(3, price);
ps.setString(4, description);  ps.setString(5, categoryId[0]);
ps.executeUpdate();
```

Order module (`ManageOrder`). This is the main module. The user selects a customer and a product from two tables, enters the quantity and clicks Add To Cart; the code checks the available stock, rejects a duplicate product, calculates the sub-total and shows the running total. Clicking a cart row removes it after confirmation. When the order is saved, the stock of each cart item is reduced, the order is inserted into orderDetail with a unique bill number generated using System.nanoTime(), and a PDF is written to a file using FileOutputStream, PdfWriter, PdfPTable and PdfPCell of iText. OpenPdf.OpenById() then opens the file.

```
PdfWriter.getInstance(doc, new FileOutputStream(
        InventoryUtils.billPath + "" + orderId + ".pdf"));
doc.open();
PdfPTable tb1 = new PdfPTable(5);   // Name, Description, Price, Quantity, Sub
Total
for (int i = 0; i < tableCart.getRowCount(); i++) {
    tb1.addCell(tableCart.getValueAt(i, 1).toString());
    ...
}
doc.add(tb1);
```

View Order module (`ViewOrder`, `common.OpenPdf`). The customers are listed in one table; clicking a customer queries orderDetail for that customer and shows his or her orders (order id, date, total) in the second table. Clicking an order calls OpenPdf.OpenById(orderId), which checks whether the file exists and opens it with the Windows rundll32 command or shows a “File Does not Exist” message.

Mapping to the syllabus. The Java features used in the project are summarised in Table 5.2.

Table 5.2 Java concepts used

Concept	Where it is used
Classes and objects	Every screen (Login, Home, ManageProduct, ...) and the Connection, Statement, PreparedStatement and ResultSet objects
Inheritance	All screen classes extend javax.swing.JFrame
Polymorphism / overloading	Constructor overloading (Home() and Home(String role)); event listeners called through the ActionListener and MouseListener interfaces
Packages	dao, common, inventory.management.system
Exception handling	try ... catch ... finally in all database and file operations; NumberFormatException possible on number parsing
Multithreading	Swing GUI is started on the event dispatch thread using EventQueue.invokeLater()
Swing and events	JFrame, JTable, JComboBox, JOptionPane; ActionEvent and MouseEvent handlers
File I/O	FileOutputStream and File for creating and checking PDF bills
JDBC	DriverManager, PreparedStatement, ResultSet, SQL insert, update, select and inner join

5.2 Testing

5.2.1 Unit Testing

Unit testing checked the validation logic of the modules, including invalid inputs and exception cases. The regular expressions used in ManageUser, ManageCustomer, ManageProduct and Login were executed in Java with the sample inputs below, and the actual results are recorded in Table 5.3.

Table 5.3 Unit test cases for input validation

Test ID	Test description	Input	Expected output	Actual output	Result
U1	Name accepts letters	Ram Sharma	Accepted	Accepted	Pass
U2	Name rejects digits	Ram123	Rejected	Rejected	Pass
U3	Valid mobile number	9841234567	Accepted	Accepted	Pass
U4	Mobile not starting with 97/98	9641234567	Rejected	Rejected	Pass
U5	Mobile with fewer than 10 digits	98412345	Rejected	Rejected	Pass
U6	Valid e-mail (login)	admin@mail.com	Accepted	Accepted	Pass

Test ID	Test description	Input	Expected output	Actual output	Result
U7	E-mail without @	admin.mail.com	Rejected	Rejected	Pass
U8	Valid e-mail (customer)	ram@mail.com	Accepted	Accepted	Pass
U9	E-mail without domain suffix (customer)	ram@mail	Rejected	Rejected	Pass
U10	Product name with letters and digits	Keyboard 101	Accepted	Accepted	Pass
U11	Product name with special character	Key-board	Rejected	Rejected	Pass
U12	Valid quantity	25	Accepted	Accepted	Pass
U13	Negative quantity	-5	Rejected	Rejected	Pass
U14	Non-numeric quantity	abc	Rejected	Rejected	Pass
U15	Empty quantity	(blank)	Rejected	Rejected	Pass

Test ID	Test description	Input	Expected output	Actual output	Result
U16	Valid decimal price	499.50	Accepted	Accepted	Pass
U17	Invalid price	49x	Rejected	Rejected	Pass
U18	Non-numeric order quantity in Add To Cart	abc	Error message	Parsing throws NumberFormatException; no message shown to the user	Fail

Test U18 shows an exception case that is not handled: in ManageOrder the quantity is converted with Integer.parseInt() outside the try block, so a non-numeric quantity raises an exception instead of a message. This is listed as an improvement in Section 6.2.

5.2.2 System / Integration Testing

System testing checks the complete flow with all modules working together on a computer that has the MySQL database and the project libraries installed. Table 5.4 lists the test cases prepared for the full flow. The **Actual output** and **Result** columns are completed by the team when each test is executed on the final build.

Table 5.4 System / integration test cases

Test ID	Test description	Input	Expected output
S1	Login with valid active user	Valid e-mail and password	Home screen opens

Test ID	Test description	Input	Expected output
S2	Login with wrong password	Valid e-mail, wrong password	"Incorrect Email or Password"
S3	Login with Inactive user	Inactive user's credentials	"Incorrect Email or Password"
S4	Add category, then add product in it	Category "Electronics"; product "Keyboard"	Product appears in table with category name
S5	Add customer and view in Order screen	Valid customer details	Customer listed in both screens
S6	Order quantity above stock	Quantity greater than stock	"Product is out of Stock. Only N Left"
S7	Add product twice to cart	Same product again	"Product already exist in Cart"
S8	Save order with valid cart	1 customer, 2 products	Stock reduced, order stored, PDF bill opened
S9	View order of customer	Select customer, click order	Order list shown; PDF bill opens
S10	Admin login (non-Super Admin)	Admin credentials	User button hidden on Home
S11	Logout	Click Logout, confirm Yes	Login screen shown

5.3 Result Analysis

All input-validation rules tested in Table 5.3 behave as designed, which satisfies the objective of validated data entry. From the implementation, the modules provide login

with roles, management of users, categories, products and customers, stock checking, cart and total calculation, automatic stock deduction, order saving and PDF bill generation, which covers the objectives listed in Section 1.3. The one failed case (U18) shows that exception handling around numeric input must be improved. The system test cases in Table 5.4 should be run and recorded on the final build before submission to confirm that the complete flow meets the objectives.

CHAPTER 6: CONCLUSION AND FUTURE RECOMMENDATIONS

6.1 Conclusion

This project developed an Inventory Management System in Java that replaces manual stock and billing records with a database-driven desktop application. It provides role-based login, management of users, categories, products and customers, an order module with a cart that checks and reduces stock automatically, a customer-wise order history and PDF bills. In this way it addresses the problems of time consumption, redundancy, calculation errors, poor searching and lack of access control stated in Chapter 1.

While building the system the team applied the key Java concepts of the course: classes and inheritance with JFrame, constructor overloading, packages, Swing components and event handling, exception handling, regular-expression validation, file output with iText and JDBC with PreparedStatement and SQL joins. The team also learned to follow the software development life cycle with UML modelling, database design and testing.

6.2 Future Recommendations

a) **Security:** store passwords as salted hashes (for example SHA-256 or bcrypt), use PreparedStatement for the login query as well, and keep database credentials in a configuration file instead of the source code.

b) **Reliability:** wrap the order saving (stock update and order insert) in a database transaction, add FOREIGN KEY constraints, and handle non-numeric input without exceptions.

c) **More features:** delete and search options, low-stock alerts, supplier and purchase-order modules, discounts and tax in the bill, and storing order items in a separate table.

d) **Reports:** export reports to PDF or Excel, such as daily sales and current stock.

e) **Platform:** convert to a web application with Servlet/JSP or Spring Boot, or to a mobile application, and move the database to the cloud for multi-branch use.

REFERENCES

[1] S. Axsäter, Inventory Control, Springer New York, 2000.

[2] J. W. Toomey, Inventory Management: Principles, Concepts and Techniques, Springer New York, 2024.

[3] J. Mapes, Inventory Control Systems, John Wiley & Sons, Inc. (Wiley Encyclopedia of Management), 2020.

[4] S. Axsäter, Inventory Control, 3rd Edition, Springer Cham / Springer International Publishing Switzerland, 2015.

[5] C. D. K. Snook, Inventory Management: Principles and Practices, Routledge, 2019.

[6] Z. C. P. Ltd., Zoho Inventory – Online Inventory Management Software, Zoho Corporation, 2026.

[7] O. S.A., Odoo Inventory – Inventory Management Software, Odoo S.A., 2026.

APPENDICES

Appendix A — Screenshots of Major Screens

The figures below show the layout of every screen of the final system, rendered from the Swing form definitions of the project (component positions, labels and background images). Replace them with real screenshots from the running application before final submission if required.



Figure A.1 Login screen



Figure A.2 Home screen (dashboard)

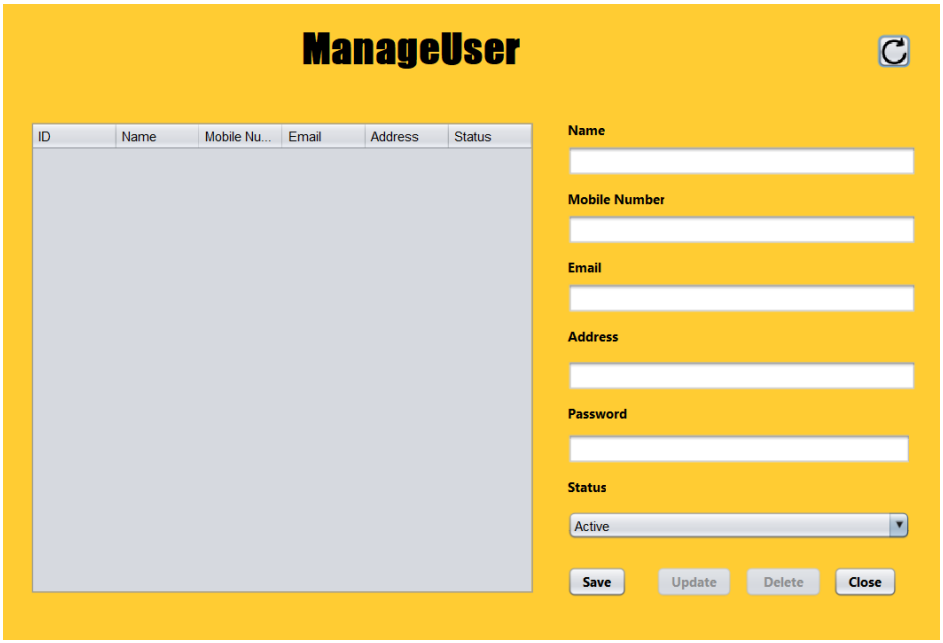


Figure A. 3 Manage User screen

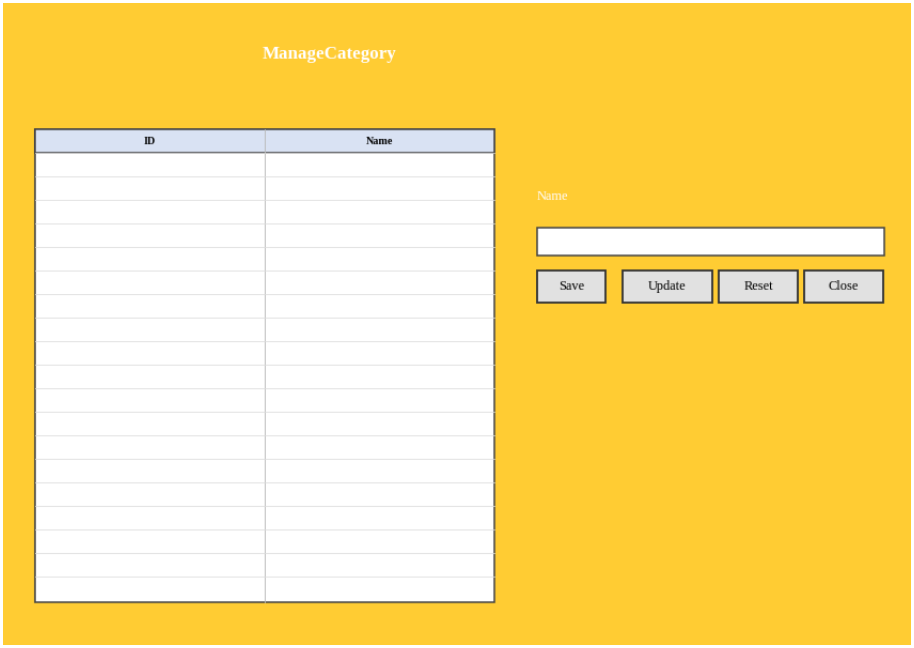


Figure A.4 Manage Category screen

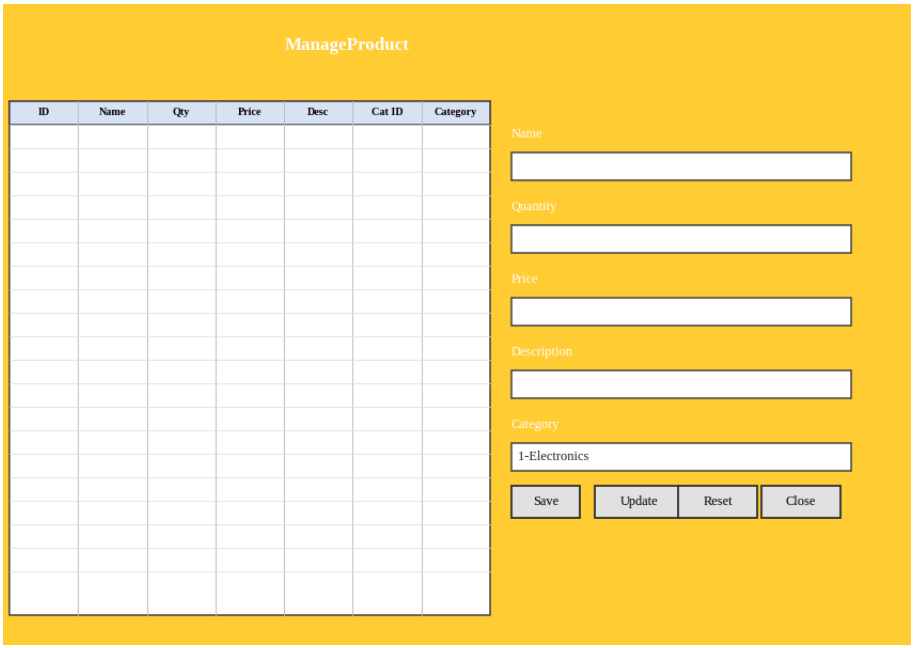


Figure A.5 Manage Product screen

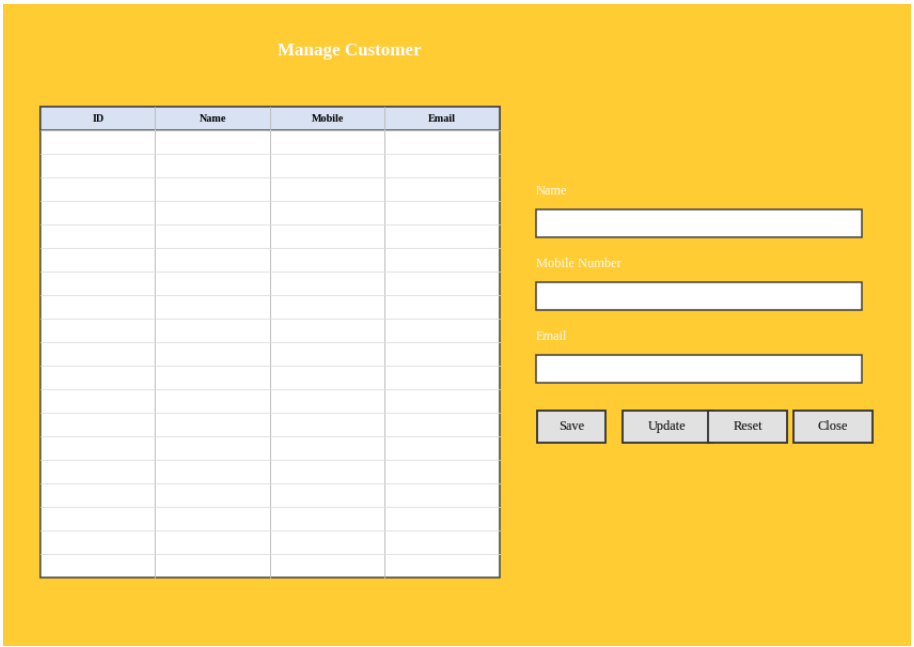


Figure A.6 Manage Customer screen

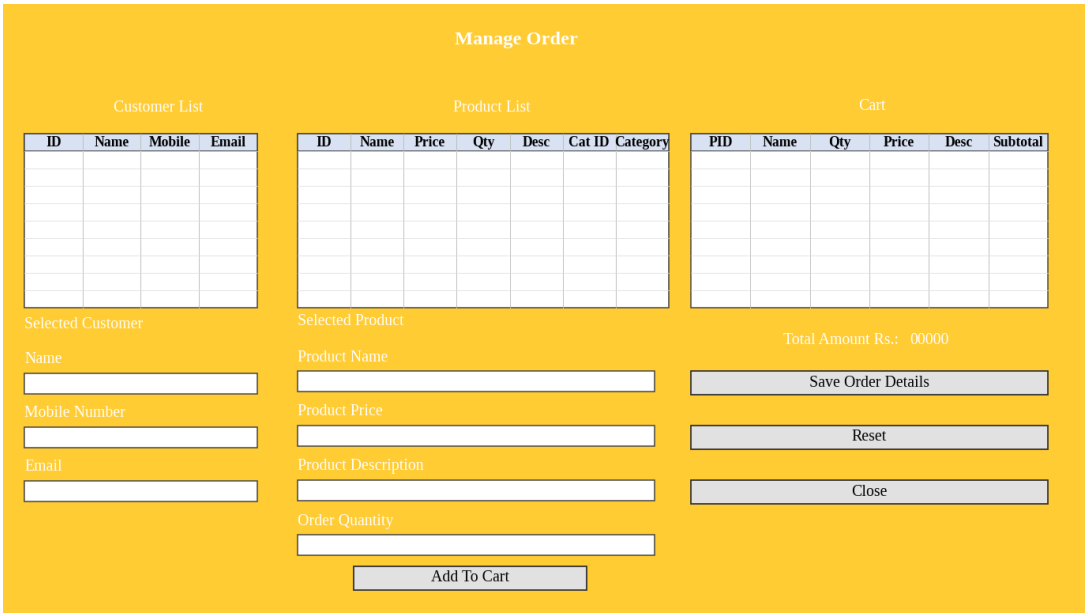


Figure A.7 Manage Order screen

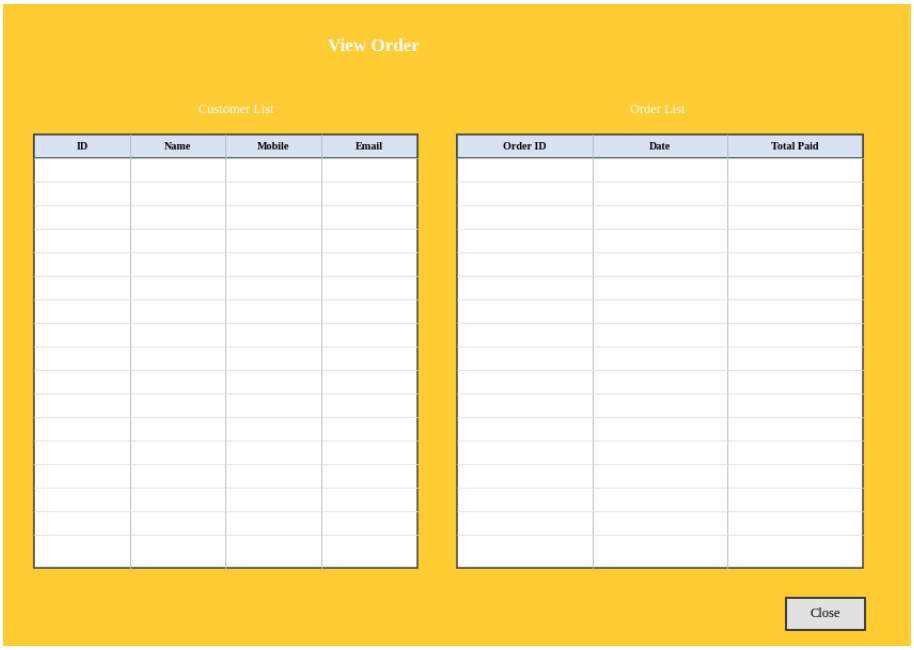


Figure A.8 View Order screen

Appendix B — Source Code of Key Classes

B.1 dao/ConnectionProvider.java

```
package dao;
import java.sql.Connection;
import java.sql.DriverManager;

public class ConnectionProvider {
    public static Connection getCon(){
        try{
            Class.forName("com.mysql.cj.jdbc.Driver");
            Connection con = DriverManager.getConnection(

"jdbc:mysql://localhost:3306/inventory?useSSL=false","root","********");
            return con;
        }
        catch(Exception e){
            System.out.println(e);
            return null;
        }
    }
}
```

B.2 Login.java – login button handler

```
private void jButton2ActionPerformed(java.awt.event.ActionEvent evt) {
    String email = txtEmail.getText();
    String password = txtPassword.getText();
    if (!email.matches("^[A-Za-z0-9+_.-]+@[A-Za-z0-9.-]+$")) {
        JOptionPane.showMessageDialog(null, "Please enter a valid email address");
        return;
    }
    int temp = 0;
    try {
        Connection con = ConnectionProvider.getCon();
```

```
        Statement st = con.createStatement();
        ResultSet rs = st.executeQuery("select *from appuser where email='" + email
            + "' and password='" + password + "' and status='Active'");
        while (rs.next()) {
            temp = 1;
            setVisible(false);
            new Home(rs.getString("userRole")).setVisible(true);
        }
        if (temp == 0) {
            JOptionPane.showMessageDialog(null, "Incorrect Email or Password");
        }
    } catch (Exception e) {
        JOptionPane.showMessageDialog(null, e);
    }
}
```

B.3 ManageUser.java – input validation

```
private boolean validateUserInput() {
    String name = txtName.getText().trim();
    String mobileNumber = txtMobileNumber.getText().trim();
    String email = txtEmail.getText().trim();
    if (!name.matches("[a-zA-Z ]+")) {
        JOptionPane.showMessageDialog(null, "Name must contain alphabets only.");
        txtName.requestFocus();  return false;
    }
    if (!mobileNumber.matches("^(97|98)[0-9]{8}$")) {
        JOptionPane.showMessageDialog(null,
            "Mobile number must start with 97 or 98 and contain exactly 10 digits.");
        txtMobileNumber.requestFocus();  return false;
    }
    if (!email.matches("^[A-Za-z0-9+_.-]+@[A-Za-z0-9.-]+$")) {
        JOptionPane.showMessageDialog(null, "Please enter a valid email address.");
        txtEmail.requestFocus();  return false;
    }
```

```
    return true;
}
```

B.4 ManageOrder.java – add to cart

```
String noOfUnits = txtOrderQuantity.getText();
if (!noOfUnits.equals("")) {
    String productName = txtProductName.getText();
    String productPrice = txtProductPrice.getText();
    int totalPrice = Integer.parseInt(txtOrderQuantity.getText()) *
Integer.parseInt(productPrice);
    int checkStock = 0;  int checkProductAlreadyExistInCart = 0;
    try {
        Connection con = ConnectionProvider.getCon();
        Statement st = con.createStatement();
        ResultSet rs = st.executeQuery("select *from product where product_pk=" +
productPk + "");
        while (rs.next()) {
            if (rs.getInt("quantity") >= Integer.parseInt(noOfUnits)) { checkStock = 1; }
            else JOptionPane.showMessageDialog(null,
                "Product is out of Stock. Only " + rs.getInt("quantity") + " Left");
        }
    } catch (Exception e) { JOptionPane.showMessageDialog(null, e); }
    ...  // duplicate-product check, then model.addRow(...) and update total
}
```

B.5 ManageOrder.java – save order (stock update and order insert)

```
orderId = getUniqueId("Bill-");
for (int i = 0; i < tableCart.getRowCount(); i++) {
    st.executeUpdate("update product set quantity=quantity-" + qty + " where
product_pk=" + pk);
}
PreparedStatement ps = con.prepareStatement(
    "insert into orderDetail(orderId,customer_fk,orderDate,totalPaid)
values(?,?,?,?)");
```

```
ps.setString(1, orderId);  ps.setInt(2, customerPk);
ps.setString(3, myFormat.format(cal.getTime()));  ps.setInt(4, finalTotalPrice);
ps.executeUpdate();
// ... iText PDF bill is created and opened with OpenPdf.OpenById(orderId)
```

Appendix C — User Manual

1. Install the requirements. Install a JDK, Apache NetBeans and MySQL Server. Add MySQL Connector/J 8.0.20 and the iText jar to the project libraries.

2. Create the database. In MySQL run create database inventory; and then create the tables:

```
create table appuser(appuser_pk int AUTO_INCREMENT primary key, userRole
varchar(50),
 name varchar(200), mobileNumber varchar(50), email varchar(200),
 password varchar(50), address varchar(200), status varchar(50));
create table category(category_pk int AUTO_INCREMENT primary key, name
varchar(200));
create table product(product_pk int AUTO_INCREMENT primary key, name
varchar(200),
 quantity int, price int, description varchar(500), category_fk int);
create table customer(customer_pk int AUTO_INCREMENT primary key, name
varchar(200),
 mobileNumber varchar(50), email varchar(200));
create table orderDetail(order_pk int AUTO_INCREMENT primary key, orderId
varchar(200),
 customer_fk int, orderDate varchar(200), totalPaid int);
```

3. Insert the first user. Insert a Super Admin so that the first login is possible:

```
insert into appuser (userRole,name,mobileNumber,email,password,address,status)
values ('SuperAdmin','Super
Admin','98415','superadmin@testemail.com','admin','Nepal','Active');
```

4. Configure the project. In dao/ConnectionProvider.java set your MySQL user name and password. In dao/InventoryUtils.java set billPath to an existing folder where the PDF bills will be saved (default D:\).

5. Run. Open the project in NetBeans, run Login.java, log in with the Super Admin e-mail and password, and change the default password. Add categories first, then products and customers, then create orders from the Order screen. Use View Order to see the order history and open bills.